

Is A Model Actually Any Good and Why?

So far, you have fitted some models and given them a score.

Today you learn which score to use and trust. More importantly, when a good score is hiding a useless model.

- → or Space — next point
- ← — back · Home / End — jump
- The sidebar tracks where you are

Ready? Press → to begin. Deck built 2026-09-21.

Thinking Time!

What do you know?

You have fitted a model that forecasts rain from temperature, humidity and wind speed. It runs. It can make predictions. But you ask yourself, *“is it any good?”*

- What is a good model and how can you measure it?
- Your model's accuracy is 90%. Is that good?
- Your model forecasts “no rain” in a desert. It is correct 99% of the time. Is that a good weather model?
- How do you or your model know which mistakes are more costly?

Fitting a model can be done quickly and easily. Grading it is the hard part.

What we will learn

Nine Metrics, One Question

Every metric provides a means for us to evaluate a model. We need to choose the right one for our specific use case.

- Build a **confusion matrix** for classification tasks
- Show how **accuracy** can be deceiving
- Fit a **baseline** model before you trust any other model score
- Discuss how **recall** relates to **precision** and how they can be traded off
- Look at the **F1** score and sweep decision thresholds
- Revise our understanding of: **MSE, RMSE, MAE and R²**

Let's go back to the wine dataset.

Our Data

4,898 Bottles of White On The Wall...

Eleven physical measurements per bottle, plus a tasting panel’s quality score from 3 to 9.

wine-quality.csv — the first four bottles, all eleven features and the target

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
7.0	0.27	0.36	20.7	0.045	45	170	1.001	3.00	0.45	8.8	6
6.3	0.30	0.34	1.6	0.049	14	132	0.994	3.30	0.49	9.5	6
8.1	0.28	0.40	6.9	0.050	30	97	0.995	3.26	0.44	10.1	6
7.2	0.23	0.32	8.5	0.058	47	186	0.996	3.19	0.40	9.9	6

Eleven columns of chemistry · one column, `quality` , scored by people. That last column is the only thing we are trying to predict.

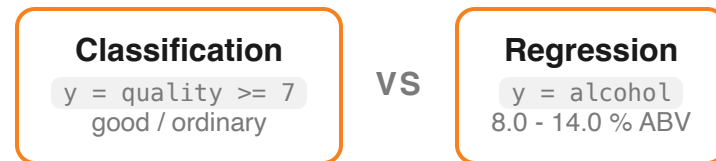
- A wine is **good** when `quality >= 7` .
- Calculated, that is **1,060** bottles out of **3,838** = **21.6%**
- Keep that 21.6% in your head, it’s important later.
- We now have a classification problem. Good ≥ 7 , Regular < 7 : if you guessed “regular” for every bottle, how accurate would you be? Put a number in the chat.

Let’s look more closely at classification problems.

Both regression problems, just different targets

A Number Or A Category?

The metric you reach for is decided before you fit any model, by the shape of what you are predicting.



- A classification problem is either *right or wrong, yes or no, true or false, 1 or 0* and we score it four ways.
- A regression problem is *off by some amount*, so we measure the size of the miss (the error).
- The type of problem determines the appropriate evaluation metrics.

Categories first.

Four Ways To Be Right Or Wrong

Every prediction a classifier makes lands in exactly one of four quarters.

Rows are the truth · columns are the prediction

	predicted ordinary	predicted good
actually ordinary	TN — True Negative	FP — False Positive
actually good	FN — False Negative	TP — True Positive

```
from sklearn.metrics import confusion_matrix
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000) # classifier. We cover this on Monday
model.fit(X_train, y_train)               # 3,673 training bottles
y_pred = model.predict(X_test)            # 1,225 held-out bottles
print(confusion_matrix(y_test, y_pred))

# [[912,  48],    · 912 ordinary wines called ordinary, 48 called good
# [188,  77]]    · 188 good wines MISSED, 77 good ones found
```

- sklearn orders the confusion matrix as `[[TN, FP], [FN, TP]]` — truth down the side, prediction across the top.
- The mistake everyone makes once: **reading it transposed**. Most textbooks, and plenty of blog posts, draw it the other way round — prediction down the side, truth across the top.
- Read it mirrored and the 48 false positives become 48 false negatives. Precision and recall then swap places, and every conclusion you draw from them is backwards.
- So check one cell you can reason about before trusting the other three. Most wines are ordinary and the model mostly says ordinary, so the largest count has to sit top-left: **912** confirms that the rows really are the truth.
- **FN = 188**. The model missed 188 of the 265 good wines in the test set = 71%.

Four numbers — let's see how we can use them.

Accuracy

80.7% Accurate Can Be Deceiving

Accuracy is the obvious metric for every prediction. How accurate is our model — how many did we get right?

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{TN} + \text{FN})$$

TP true positive · TN true negative · FP false positive · FN false negative

```
from sklearn.metrics import accuracy_score

accuracy_score(y_test, y_pred)  # 0.807 strong start!
```

- 80.7% correct. Would you use it?

Now fit the dumbest model that could possibly work.

```
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="most_frequent")  # always says "ordinary" wine
baseline.fit(X_train, y_train)
baseline.score(X_test, y_test)  # 0.784 – it never looks at the features
```

- A model that ignores every feature scores **0.784**. Ours scores **0.807**.
- All that work bought **2.3 percentage points**. This is the **accuracy paradox**: when one class dominates, accuracy mostly measures the class imbalance.
- Fit a **baseline model** *first*. A score means nothing until you know what you could get for free.

The model found only 77 of the 265 good wines — we need a different metric that captures this.

Recall

How Many Good Wines Did We Catch?

Recall ignores the majority class entirely. It asks: “of everything that was labelled *good*, what fraction did we find?”

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

TP true positive · FN false negative — the positives we missed

```
from sklearn.metrics import recall_score  
  
recall_score(y_test, y_pred)  # 0.291 – 77 found / 265 actually good
```

- Accuracy said 0.807. Recall says **0.291**. Same model, same predictions, different perspectives.
- The model missed **71%** of the good bottles. Accuracy never showed you that, because it was drowned out by the majority “ordinary” wine class.
- Recall is also called *sensitivity*, or the *true positive rate*.

Recall is easy to trick, though: label every bottle good and recall hits 1.0. We need the other side of the story.

Precision

When We Said Good, Were We Right?

Precision looks only at the predictions you act on. Of every wine flagged good, what fraction really was?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

TP true positive · FP false positive — the false alarms

```
from sklearn.metrics import precision_score  
  
precision_score(y_test, y_pred)    # 0.616 - 77 correct / 125 flagged good
```

- The model flagged 125 bottles. 77 were genuinely good, 48 were not.
- Recall counts what you *missed* (FN). Precision counts what you *falsely raised* (FP).

Which mistake is more costly? Which metric is better suited?

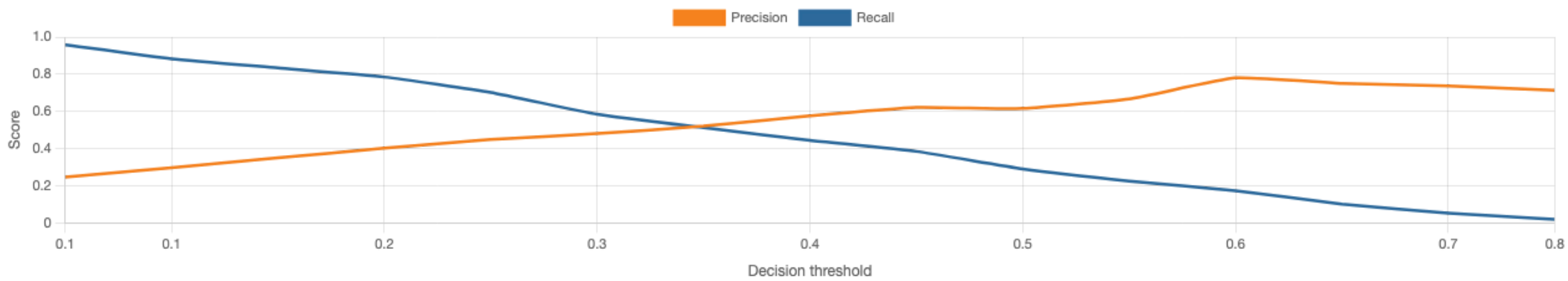
- Cancer screening — miss a tumour, or scare a healthy patient?
- Spam filter — let junk emails through, or bin a real invoice?

You can trade one for the other without refitting your model.

Dial in on Your Preferences

A classifier does not output a class. It outputs a probability, and a threshold turns that probability into a “yes” or a “no” — typically at 0.50. Anything above the threshold is classified as “yes”; anything below is classified as “no”.

```
proba = model.predict_proba(X_test)[: , 1] # column 1 = P(good)
y_pred_050 = (proba >= 0.50).astype(int) # the default
y_pred_030 = (proba >= 0.30).astype(int) # more willing to say the wine is "good"
```



Every threshold from 0.05 to 0.75 on the 1,225 test bottles. Orange = precision | Blue = recall.

The same model, two thresholds — nothing was refitted

threshold	precision	recall	F1
0.50	0.616	0.291	0.395
0.30	0.523	0.654	0.584

- The closer the score is to 1, the better. Dropping the threshold to 0.30 lifts recall from 0.291 to **0.654**, but it costs precision: 0.616 down to 0.523.
- The curves cross over at 0.36. There is no setting where both are high; that is the trade-off, not a bug.

0.50 is a default threshold, not a law. Pick it from which mistake costs you more.

F1-score

One Number That Balances Precision and Recall

When you must rank models on a single score but refuse to ignore either error, combine them.

$$F1 = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

Precision fraction of flagged positives that were right · Recall fraction of real positives that were found

```
from sklearn.metrics import f1_score

f1_score(y_test, y_pred)  # 0.395 - at the default 0.50 threshold
```

- F1 is the **harmonic mean**, not the ordinary mean. The plain mean of 0.616 and 0.291 would be 0.454.
- The harmonic mean is dragged toward the *smaller* of the two values, so one terrible score cannot be outweighed by an excellent one.
- Precision = 1.0 and Recall = 0.0 makes F1 = 0.0. Exactly the behaviour you want from a summary of the two metrics.

F1 judges one threshold. Next, we judge every threshold at once...

Receiver Operating Characteristic (ROC) Curve

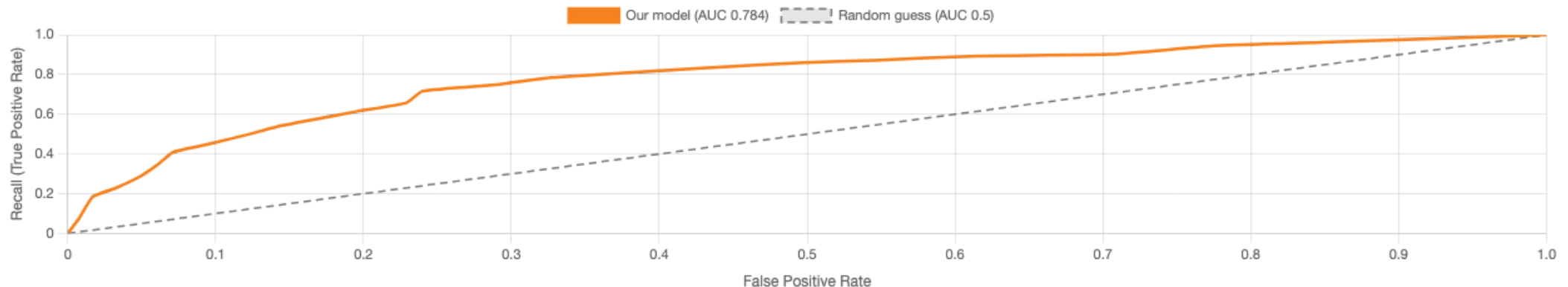
Sweeping the thresholds from 0 to 1, plot the false positive rate against recall and you get a ROC curve.

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

FPR — false positive rate: the share of ordinary wines wrongly flagged as good

```
from sklearn.metrics import roc_curve, roc_auc_score

proba = model.predict_proba(X_test)[: , 1]          # P(good)
fpr, recall, thresholds = roc_curve(y_test, proba)
roc_auc_score(y_test, proba)                       # 0.784
```



- A common mistake to avoid: passing `y_pred` instead of `proba`, because the ROC curve requires probability scores, not binary predictions.
- The dashed diagonal is the “no skill” baseline — 0.5 is the score you get by guessing at random. **AUC** is the area under your curve: 1.0 is a perfect score.
- An AUC of 0.784: pick one “good” and one “ordinary” bottle of wine at random, and the model scores the good one higher 78.4% of the time.

0.784 looks respectable, but we need to be cautious about interpreting this metric on imbalanced data.

All data scientists find imbalanced datasets challenging

So they use an accuracy metric that considers both classes. Only 21.6% of the wine bottles are good. Two tools stop the majority class flattering the score.

$$\text{Balanced accuracy} = (\text{Recall}(-ve) + \text{Recall}(+ve)) / 2$$

Recall(+ve) — recall on the positive class · Recall(-ve) — recall on the negative class · each class counts equally, whatever its size

```
from sklearn.metrics import accuracy_score, balanced_accuracy_score

accuracy_score(y_test, y_pred)      # 0.807
balanced_accuracy_score(y_test, y_pred)  # 0.620

# The dummy classifier that always predicts the majority class scores 0.500
```

- The drop from 0.807 → **0.620** is the overestimated accuracy that was given to the model.

We can now plot the precision-recall curve:



- ROC's “no skill” line is the diagonal, and it uses the FPR. The precision-recall “no skill” line is the **positive rate** = the number of positive instances / total instances (0.216).
- The FPR divides by 3,838 ordinary wines, so a few hundred false positives barely move it. **ROC overestimates a model whenever the negative class is disproportionately larger**. Precision divides by what was actually flagged, and cannot hide them.

Imbalanced data? Rare positives? Read the precision-recall curve.

Let's add to our regression metrics

MAE, MSE, and RMSE

Let's try to predict the alcohol content of a wine. Same 4,898 bottles, new target: `alcohol`, a number from 8.0 to 14.0 % ABV. Nothing is right or wrong any more — only near or far (in terms of error). So we measure the size of the miss.

$$\text{MAE} = (1/n) \cdot \sum |y_i - \hat{y}_i|$$

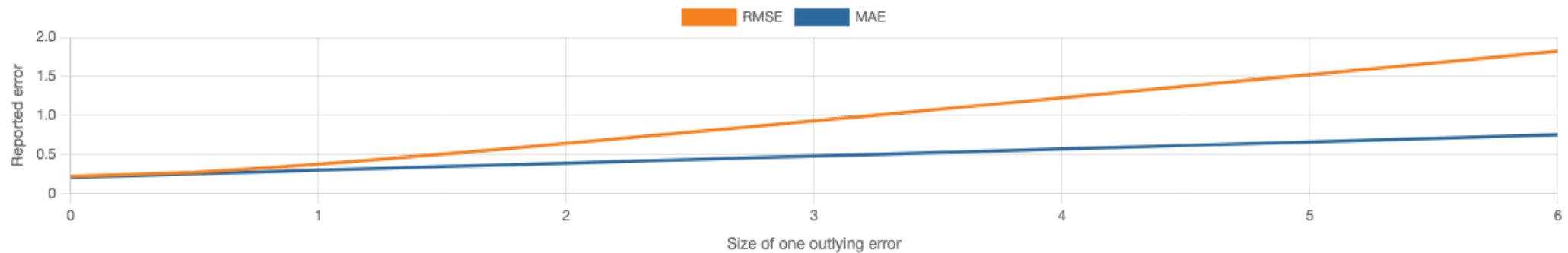
n number of observations · y_i actual value · $\hat{y}_i$ predicted value

$$\text{MSE} = (1/n) \cdot \sum (y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

the square punishes a big miss far harder than two small ones · the root puts the answer back into target units (% ABV)

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error

mean_absolute_error(y_test, y_pred)    # 0.292 % ABV · the typical miss
root_mean_squared_error(y_test, y_pred) # 0.382 % ABV · bigger. always.
```



- MAE rises steadily, in an almost straight line; RMSE takes off.
- So **RMSE > MAE, always**. But an RMSE that dwarfs MAE diagnoses unevenly distributed errors: your predictions suffer from a few massive outliers rather than small, uniform misses. Here, 0.382 against 0.292 implies a minority of bottles are badly mispredicted.

Add this to what you learned on Monday.

R²

Compared To Just Guessing The Mean

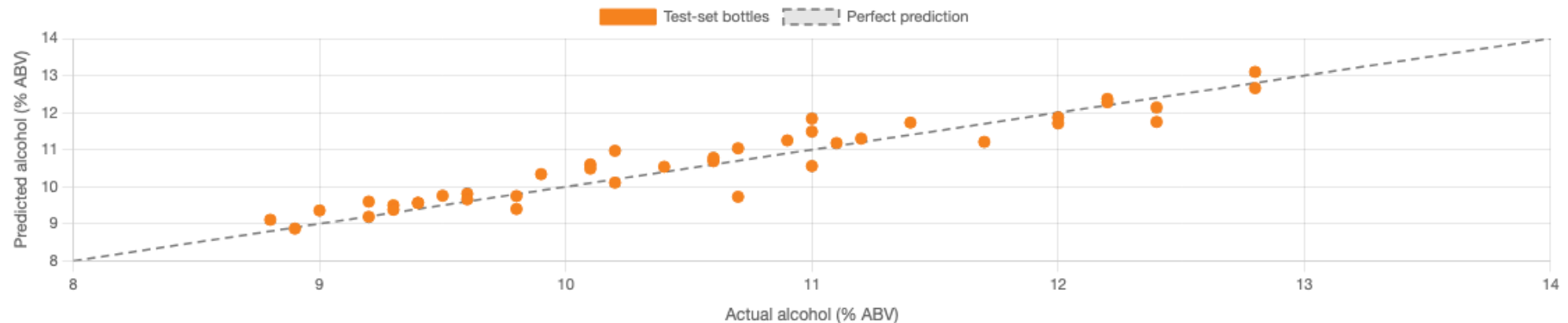
An error of 0.292 % ABV means nothing until you know what a thoughtless model would score.

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

SSres sum of squared residuals, $\sum (y_i - \hat{y}_i)^2$ · SStot total sum of squares, $\sum (y_i - \bar{y})^2$ where $\bar{y}$ is the mean

```
from sklearn.metrics import r2_score

r2_score(y_test, y_pred)    # 0.904
# DummyRegressor(strategy="mean") scores -0.0 here, by construction
```



- R² is the variance the model explains that the mean does not. **0.904** is a seemingly good fit, but it needs adjusting.
- Always-guessing-the-mean scores **0.0** — not by luck, by definition. That is why `DummyRegressor` is the free baseline here.
- R² *can* go negative. That means the model did worse than the mean, and it is a real thing to see on test data. Any result is a good result.

Nine metrics, too many to choose from... so when do we use each one?

There Is No Best Metric

The right metric depends on your specific problem and the costs of different types of errors.

Read the left column as a sentence about your problem

When...	reach for	because...
missing a positive is expensive	recall	Screening mammograms: a missed tumour costs a life, a false alarm costs one more scan. Recall counts the misses.
a false alarm is expensive	precision	A spam filter: junk in the inbox is a nuisance, a client's invoice in the spam folder is a lost contract. Precision counts the mistake.
you need one number, both matter	F1	Ranking five fraud models: A missed fraud and a frozen card both cost money, and F1 refuses to let a model win on one while failing the other.
comparing models, threshold not yet fixed	ROC AUC	Choosing a churn model before marketing has decided how many customers it can afford to call. AUC scores the ranking at every threshold, so the choice survives that decision.
positives are rare	precision-recall curve, balanced accuracy	Card fraud at 2 cases in 1,000: FPR divides by the huge negative class, so ROC looks superb whatever you do. Precision divides by what you flagged and cannot.
predicting a number, important outliers	RMSE	Forecasting electricity demand: one badly missed peak day blacks out a grid, and squaring the error is what makes the metric say so.
number prediction, unimportant outliers	MAE	Predicting a delivery time in minutes: 20 minutes out is exactly twice as annoying as 10, so the plain average miss is the honest number.

Two more regression metrics worth knowing and some tricks:

```
from sklearn.metrics import mean_absolute_percentage_error, median_absolute_error

mean_absolute_percentage_error(y_test, y_pred)  # 0.028  · MAPE: 2.8% off, unit-free
median_absolute_error(y_test, y_pred)          # 0.243  · vs MAE 0.292
```

- **MAPE** is a percentage, so it compares across targets on different scales. It explodes if any actual value is near zero.
- **Median absolute error** ignores outliers completely. $0.243 < 0.292$ is the same warning RMSE conveys — a few rows are badly wrong.
- When the numbers disagree, plot the residuals with `PredictionErrorDisplay` and go look at the rows that failed.

Want everything? use `classification_report` or `regression_metrics`. But! Always pick the metric before you fit a model. Why?

Nine Metrics, One Page

Two kinds of problem, four counts, and every formula that falls out of them.

Classification
the target is a *category*
a prediction is right or wrong

VS

Regression
the target is a *number*
a prediction is near or far

Classification — the confusion matrix, and the four counts every classification metric is built from

	predicted negative	predicted positive
actually negative	TN true negative — a correct all-clear	FP false positive — a false alarm
actually positive	FN false negative — a miss	TP true positive — a catch

Six ways of dividing those four counts

Metric	Formula	What it says	Today
Accuracy	$(TP + TN) / (TP + FP + TN + FN)$	of all predictions, the share that were right	0.807
Recall	$TP / (TP + FN)$	of the real positives, the share we found	0.291
Precision	$TP / (TP + FP)$	of everything we flagged, the share that was right	0.616
F1	$2 \cdot (Precision \cdot Recall) / (Precision + Recall)$	the harmonic mean, pulled toward the weaker half	0.395
ROC AUC	area under recall plotted against $FPR = FP / (FP + TN)$	the ranking, judged at every threshold at once	0.784
Balanced accuracy	$(Recall_+ + Recall_-) / 2$	accuracy with each class weighted equally	0.620

Regression — nothing is right or wrong here, only near or far, so we measure the size of the miss

Metric	Formula	What it says	Today
MAE	$(1/n) \cdot \sum y_i - \hat{y}_i $	the typical miss, in the target's own units	0.292
MSE / RMSE	$(1/n) \cdot \sum (y_i - \hat{y}_i)^2 \cdot RMSE = \sqrt{MSE}$	squares the miss, so a few big errors dominate	0.382
R²	$1 - (SS_{res} / SS_{tot})$	the variance explained, versus always guessing the mean	0.904

TP true positive · TN true negative · FP false positive · FN false negative · Recall₊ recall on the positive class · Recall₋ recall on the negative class · n number of observations · y_i actual value · $\hat{y}_i$ predicted value · SS_{res} $\sum (y_i - \hat{y}_i)^2$ · SS_{tot} $\sum (y_i - \bar{y})^2$

None of those numbers means anything on its own. A `DummyClassifier` that ignores every feature scored **0.784** accuracy against our model's 0.807, and balanced accuracy **0.500**; `DummyRegressor` scores R² = 0.0 by construction. The **threshold** is a dial, not a law — moving it from 0.50 to 0.25 bought recall 0.291 → 0.702 and sold precision 0.616 → 0.449.

No metric is best. The right one is decided by what your model predicts and by the consequences of its mistakes.

Consider This

Six Situations, Six Different Answers

Every one of these is a real machine learning problem, and every one needs a different metric.

Call out the metric you would put in front of the person paying for the model — and say what it would cost to get it wrong.

- **Hiring.** A CV screener is 95% accurate, and 4% of applicants in the training dataset are qualified. What would “reject everyone” score? Which metric, and who pays for a false negative — the company, or the applicant who never hears back?
- **Intensive care.** A model flags patients developing sepsis. A miss can be fatal; a false alarm costs one more blood test. Which metric?
- **Card fraud.** A bank auto-blocks transactions it thinks are stolen. Two in a thousand really are, and a wrongly blocked card loses a customer. Which metric, and which of the two curves?
- **Churn.** Two models, and marketing has not yet decided how many customers it can afford to target. Which metric lets you pick a model before the threshold is chosen?
- **House prices.** A handful of mansions cost twenty times the median. Which metric and does your answer change if those mansions are the clients paying for the model?
- **Delivery times.** The app predicts arrival in minutes, and being 20 minutes out is exactly twice as annoying as 10. Which metric?

Somebody chose the metric on every dashboard you will ever be shown. Choosing it is a decision about people, not statistics.

Look It Up

Classification — a task whose target is a category, so a prediction is simply right or wrong.

Regression — a task whose target is a number, so a prediction is near or far and the size of the miss is what we measure.

Positive class — the outcome you are trying to detect. Naming it is a modelling choice, and it decides what TP, FP and FN mean.

Class imbalance — one class far outnumbering the other, so a model can score well by ignoring the rare one.

Confusion matrix — the 2x2 table of TP, FP, TN, FN. In scikit-learn the truth runs down the side and the prediction across the top.

True positive (TP) — predicted positive, and it really was positive. A catch.

False positive (FP) — predicted positive, actually negative. A false alarm.

True negative (TN) — predicted negative, and it really was negative. A correct all-clear.

False negative (FN) — predicted negative, actually positive. A miss.

Accuracy — of all predictions, the share that were right.

Accuracy paradox — when one class dominates, accuracy mostly measures the class balance rather than the model.

Baseline — the dumbest defensible model, fitted first, so a score has something to beat.

Recall — of the real positives, the share found. Also called sensitivity, or the true positive rate.

Precision — of everything flagged positive, the share that really was.

Decision threshold — the probability cut-off, 0.50 by default, that turns a predicted score into a predicted class.

F1-score — the harmonic mean of precision and recall.

Harmonic mean — an average pulled toward the smaller value, so one weak half cannot hide behind a strong one.

ROC curve — recall against false positive rate, traced across every threshold.

False positive rate (FPR) — of the real negatives, the share wrongly flagged positive.

ROC AUC — area under the ROC curve. 0.5 is guessing, 1.0 is perfect.

Balanced accuracy — the mean of the per-class recalls, so each class counts equally whatever its size.

Precision-recall curve — precision against recall; the honest curve when positives are rare.

Positive rate — the share of all observations that are positive. It is the no-skill line of a precision-recall curve.

MAE — mean absolute error, the typical miss, in the target's own units.

MSE — mean squared error; squares the miss, so big errors dominate.

RMSE — the root of MSE, back in the target's units. Always at least MAE.

R² — the share of variance explained versus always guessing the mean. 0 is the mean, negative is worse.

MAPE — mean absolute percentage error; unit-free, but unstable when any actual value is near zero.

Median absolute error — the middle miss; ignores outliers entirely.

Quick Quiz

1. A bottle was actually good, and the model predicted ordinary. Which cell does it land in?

- False positive
- False negative
- True negative

3. You are picking bottles for a premium line. Missing a genuinely good one costs far more than occasionally shipping an ordinary one. Which metric do you optimise?

- Precision — you must not ship a bad bottle
- Accuracy — it balances both errors
- Recall — a missed good bottle is the expensive error

5. Why compare two classifiers on ROC AUC instead of their accuracy at a 0.50 threshold?

- AUC judges the ranking across every threshold, so the comparison does not depend on a cut-off you have not chosen yet
- AUC is unaffected by class imbalance, unlike every other metric
- AUC is simply accuracy computed on probabilities instead of labels

7. A DummyClassifier using the most frequent class scores 0.784; your model scores 0.807. What is the right conclusion?

- Eleven features bought 2.3 points over guessing — investigate before claiming the model works
- The model is overfitting and needs regularisation
- The baseline is broken; a dummy model should score near zero

9. Only 21.6% of your bottles are positive. Which curve gives the more honest picture?

- The ROC curve, because it is independent of the threshold
- Either — they contain exactly the same information
- The precision-recall curve — with a large negative class, FPR stays small whatever the model does, so ROC flatters it

2. Your classifier scores 0.807 accuracy on data where 78.4% of bottles are ordinary. What have you learned?

- Very little — most of that score comes from the majority class, so you need to see recall and a baseline
- The model is strong and ready to use
- Accuracy is invalid for classification and should never be reported

4. Precision is 0.616 and recall 0.291, yet F1 is 0.395 rather than their average of 0.454. Why?

- F1 weights precision more heavily than recall
- It is the harmonic mean, which is pulled toward the smaller of the two
- F1 subtracts a penalty for class imbalance

6. Your regression reports MAE 0.292 and RMSE 0.382. What does the gap tell you?

- The two were computed on different test sets
- Nothing — RMSE is always about 30% above MAE
- A minority of bottles have unusually large errors, and squaring makes them dominate RMSE

8. Accuracy is 0.807 but balanced accuracy is 0.620. What does that gap mean?

- The model is underfitting and needs more features
- It performs far better on the majority class than on the minority one
- Balanced accuracy is always lower, so the gap carries no information

10. MAE is 0.292 but median absolute error is 0.243. What should you do?

- Report the median, since it is the smaller and therefore better number
- Investigate — a few large errors are pulling the mean up, so go and look at which bottles the model fails on
- Nothing; the mean and median of any error distribution always differ

Now on to you!

Prove The Formulas Yourself

You have seen every metric called from sklearn. Now write them, and check that your numbers match.

- Start with the **confusion-matrix quick demo** — ten predictions, by hand, until all four cells are obvious.
- Then the **classification-metrics notebook**: implement accuracy, recall, precision and F1 from scratch and assert them against sklearn.
- Do the same in the **regression-metrics notebook** for MAE, MSE and R^2 .
- Finish with the **baseline-models walkthrough** — make fitting a dummy the reflex before any real model.

If your function and sklearn disagree, you have found the thing you did not actually understand. That is the point.